

Parte Teórica

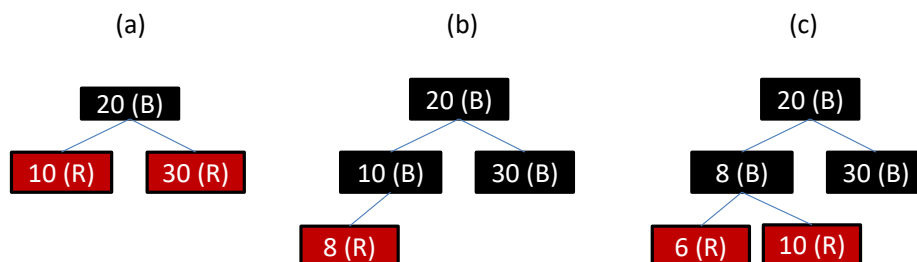
1–[3 valores] Considere o método seguinte:

```
int f(int m[]){
    int l=0, h=m.length-1;
    boolean up=true;
    while(l<h)
    {
        if (m[l]>m[h]) {
            int t=m[l];
            m[l]=m[h];
            m[h]=t;
            up=!up;
        }
        if(up)
            l++;
        else
            h--;
    }
    return h;
}
```

Indique, justificando, qual é a ordem de complexidade do método no pior caso.

2 – [3 valores] Considere a string “ARVOREHUFFMAN”. Construa uma árvore de Huffman adequada e reescreva a string com a codificação resultante.

3 – [3 valores] Indique, justificando, qual o resultado da inserção do valor “9” em cada uma das seguintes árvores RB. (*identifique em cada nodo a sua cor usando as letras “R” e “B”*).



4 – [3 valores] Considere que pretende inicializar uma *heap* binária com o conteúdo do *array* seguinte. Descreva o processo de inicialização e qual a árvore resultante (considere que o número 90 se encontra na posição 1 do *array* (o valor na posição 0 pode ser ignorado e não é representado)).

{90,9,8,7,12,71,9,71}

Parte Prática

5 – [2 valores] Construa, utilizando a classe `PriorityQueue` de forma adequada, uma classe `Arquivo` que disponibiliza um método que recebe uma factura (objecto da classe `Factura`) e outro que devolve a factura de maior valor recebida até então (e que ainda não foi devolvida). A classe `Factura` (que não precisa de construir nem pode alterar) não é comparável e disponibiliza um método `valor()`.

Para responder a a cada uma das alíneas seguintes, acrescente os métodos, variáveis ou classes internas que considerar serem necessárias.

6 – [4 valores] Considere a seguinte classe, que representa uma árvore binária

```
public class BinaryTree<T extends Comparable<? super T>>{

    static class Node<T>{
        T value;
        Node<T> left,right;
        Node(T t){value=t;left=right=null;}
    }
    // ... etc
}
```

Sabendo que é possível inicializar a árvore de indicando um comparador adequado (bem como por defeito – sem indicar qualquer informação):

- Acrescente os construtores necessários.
- Acrescente um método que devolve o menor valor da árvore.
- Acrescente um método que remove um valor da árvore.
- Acrescente um método que roda um nodo, recebido por parâmetro, com o seu descendente direito.

7 – [2 valores] Considere a seguinte classe, que representa uma árvore AVL:

```
public class AVLTree <T extends Comparable<? super T>>{
    //..
    class Node{
        T value;
        int pl,pr; //profundidade esquerda e direita
        Node left,right;

        Node(T value){
            this.value=value;
            pl=pr=0;
            left=right=nullnode;
        }
    }
}
```

- Acrescente um método que devolve a profundidade de um nodo em tempo constante.
- Acrescente o código que declara e inicializa a variável `nullnode`.
- Acrescente um método que roda um nodo, recebido por parâmetro, com o seu descendente direito (*nota: a resposta não é igual à da pergunta 6.d*).
- Acrescente um método que indica, em tempo constante, se a sub-árvore descendente de um determinado nodo recebido por parâmetro está ou não equilibrada.